

Assembler language

Introduction to the syntax

Assembler statements

- An Assembler program is made of statements
- There are four types of statements:
 1. Machine instructions. We have seen examples in the previous unit
 2. Comments. For documentation only
 3. Assembler instructions. To provide information to the Assembler
 4. Macro instructions. A compact notation for groups of statements

Assembler statements

- All Assembler statements, except comments, have this format:

1	10	16		72	73
Label	Operation	Operands	Comments	Cont.	...

- Comment statements start with an asterisk in column 1:

1				72	73
* This line is a comment statement				Cont.	...

Assembler statements - label

1	10	16		72	73
Label1	Operation	Operands	Comments	Cont.	...

- The label names the statement, so other statements can refer to it by name. For example:
 - If an instruction has the label “Calc_Tax”, you can jump to it:
J Calc_Tax
 - If a fullword has the label “Amount” you can use it in an ADD instruction:
A 3,Amount

Assembler statements - label

1	10	16		72	73
Label	Operation	Operands	Comments	Cont.	...

- The label (also called name) is optional
- If present, *it must start on column 1*
- It can be up to 63 characters long
- It is an *Assembler symbol* (details next)

Assembler statements - symbols

- The label is an *Assembler symbol*. Symbols must:
 - Start with a letter, or the characters “\$”, “_”, “#”, or “@”
 - The rest can also contain numbers
 - No other characters are allowed
 - Letters can be mixed case, but the Assembler doesn't distinguish between upper or lower case (“Hello”, “HELLO”, and “hello” are the same)

Assembler statements - symbols

- Examples of valid symbols (and so, valid labels):

A001

_start_here

HelloWorld

@handle55

- *Tip:* it is better not to use \$, @, or # in symbols, as they may look different in different countries (example: \$, in England, appears as £)

Assembler statements - symbols

- Examples of invalid symbols:

\$7462.95	Decimal point not allowed
Bond/007	Special character '/' not allowed
Set Go	No blanks allowed
Ten*Five	Asterisk not allowed
C'Wonka'	Single quotes not allowed
2Faced	Must not begin with a number

Assembler statements - operation

1	10	16		72	73
Label	Operation	Operands	Comments	Cont.	...

- Operation is *required*
- It must start at least one blank (space) from the label, if present
 - By convention, it usually starts on column 10
- It contains, for example, a machine instruction mnemonic, or an Assembler instruction

Assembler statements - operands

1	10	16		72	73
Label	Operation	Operands	Comments	Cont.	...

- Operands are optional (but often present)
- They start at least one blank from the operation
 - By convention, they usually start on column 16
- They cannot contain blanks (unless in quotes) and are separated by commas

Assembler statements - comments

1	10	16		72	73
Label	Operation	Operands	Comments	Cont.	...

- Comments (formally called *remarks*) are optional, but recommended
- They start at least one blank from the last operand
- They must end at column 71

Assembler statements - continuation

1	10	16		72	73
Label	Operation	Operands	Comments	X	...
1	10	16		72	73
		Statement continues here with operands or comments		X	...
1	10	16		72	73
		A statement can have more than one continuation			...

- **Any non-blank character** in column 72 indicates continuation
- The continuation text must start on column 16 (columns 1-15 must be blank)

Assembler statements - the rest

1	10	16		72	73
Label	Operation	Operands	Comments	Cont.	...

- The Assembler ignores anything on columns 73-80
- Note **the record ends in column 80**: historically, Assembler programs were prepared on 80-column punched cards
- Columns 73-80 were used for identifying statements with a sequence number

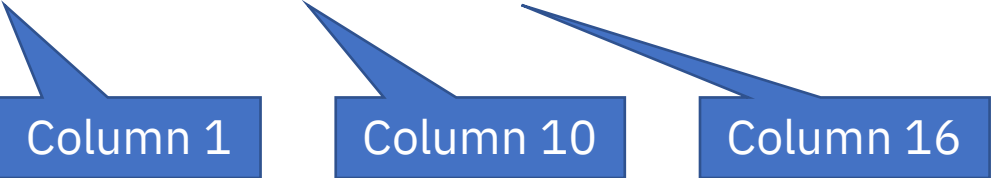
Assembler statements - example

- We will now see examples of Assembler statements:
 1. A Jump (branch) to an instruction with label **Test_Add**
 2. At **Test_Add**, an instruction that adds the contents of a fullword labelled **Integer1** to a register
 3. At label **Integer1**, the definition of a fullword
- We'll see some instructions for the first time. Don't worry about the details; we'll cover them soon

Assembler statements - example

1. Jump to an instruction with label **Test_Add**:

	J	Test_Add	Go to add
*	...		
Test_Add	A	15,Integer1	Add Integer1 to R15
*	...		



- We saw Jump in the previous unit. Recall it branches to the operand using relative addressing

Assembler statements - example

2. At **Test_Add**,
add the contents of a fullword labelled **Integer1** to a register:

Test_Add	A	15, Integer1	Add Integer1 to R15
*	...		
Integer1	DC	F'700'	Integer fullword - initial value 700
*	...		

- We also saw Add in the previous unit. It adds a fullword in memory to a register

Assembler statements - example

3. At **Integer1**, define a fullword:

Test_Add	A	15,Integer1	Add Integer1 to R15
*	...		
Integer1	DC	F'700'	Integer fullword - initial value 700
*	...		

- DC (DEFINE CONSTANT) tells the Assembler to reserve a fullword in memory with an initial value (in this case, decimal 700)
 - We'll cover DC and other Assembler instructions soon

Assembler statements - notes

- The aim of the example was to give you an idea of the syntax
- All the instructions shown will be discussed in more detail later
- DC (DEFINE CONSTANT) is an *Assembler instruction* (the others in the example are machine instructions)
- DC is used to reserve storage and fill it with an initial value
- DC is the subject of the next section